# Priority Queues

The binary heap · Algorithms § 2.4

# This video

- What a **priority queue** is, and what it's for
- Two straightforward ways to build one
    - that are unfortunately quite slow
- The **binary heap**: a tree stored in an array
- **swim** and **sink**: the methods that do the work
- The whole `MaxPQ` class and its costs

# What is a priority queue?

| collection | which item comes out next? |
|---|---|
| stack | the newest one |
| queue | the oldest one |
| **priority queue** | **the largest one (or the smallest, for a min priority queue)** |

Where priority queues show up:

- An emergency room treats the **most urgent** patient first, not whoever arrived first.
- A simulation handles whichever event happens **soonest**.
- Finding the **top 10** of a huge stream of numbers (coming up in a few slides).
- Later this term: Dijkstra's shortest paths and Prim's minimum spanning tree both use one.

# The API

```
1  public class MaxPQ<Key extends Comparable<Key>>
2
3      MaxPQ(int capacity)     // create an empty PQ
4      void insert(Key v)      // add an item
5      Key max()               // return the largest item
6      Key delMax()            // remove and return the largest
7      boolean isEmpty()       // is the PQ empty?
8      int size()              // how many items?
```

There's no separate priority number. The **item is its own priority**: it's `Comparable`, so the PQ can ask which of two items is larger.

`Key` is a placeholder type, like `Song` in the Comparable video.

`MinPQ` is the same, with `min()` and `delMin()`.

# Three different implementations

"Priority queue" names the **API**: what `insert` and `delMax` do.

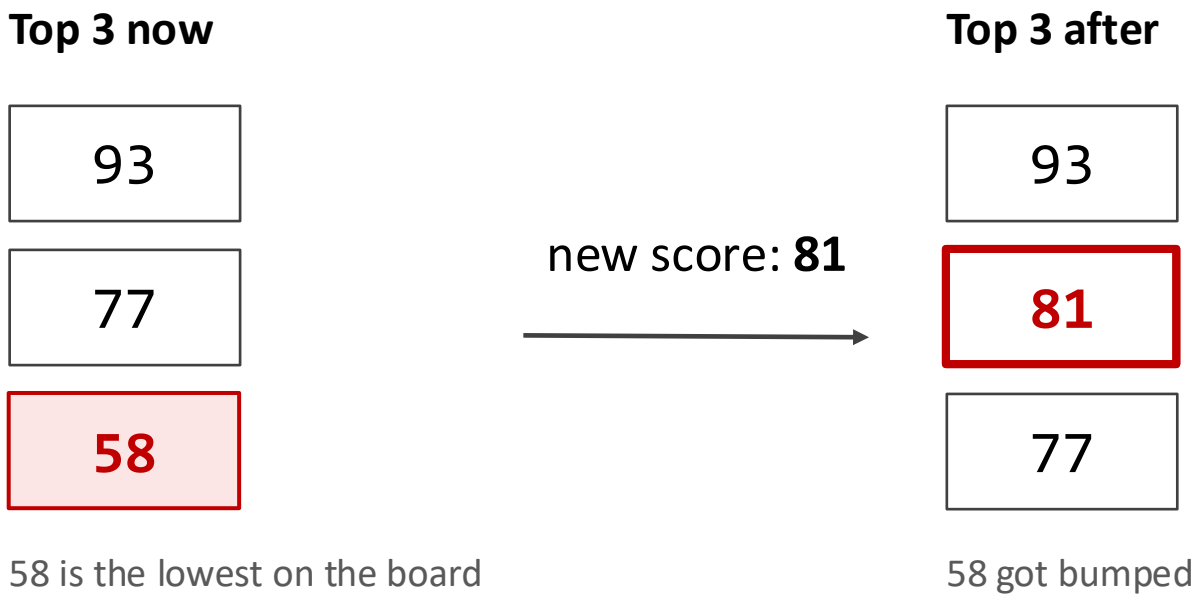
It doesn't say **how** they do it. Three ways to build one:

- an **unordered array**
- an **ordered array**
- a **binary heap** (most of this video/slide deck)

Code that **uses** a `MaxPQ` looks the same no matter which one is inside. What changes is how fast it runs.

(this is why we like interfaces; we can improve our implementation without breaking anyone's code)

# Keeping a top-3 list




To decide if a new score makes the board, we compare it with the **lowest** score on the board.

If it's bigger, the lowest score gets **bumped**.

The question we keep asking is "what's the **smallest** thing here?" That's a **min** priority queue, even though we want the **largest** items.

# A problem: the m largest of n

You have a stream of **n** numbers (let's say a billion), too many to keep. You want the **m largest** (say, the top 10).

(What's a stream? let's say you're fetching numbers one by one from the internet.)

```
Scanner in = new Scanner(System.in);
MinPQ<Integer> pq = new MinPQ<Integer>(m + 1);
while (in.hasNextInt()) {
    pq.insert(in.nextInt());
    if (pq.size() > m) {
        pq.delMin();     // throw out the smallest
    }
}
// pq now holds the m largest numbers
```

Keep a **min** priority queue that never holds more than **m** items.

When it gets too big, throw out the **smallest**: it can't be in the top m any more.

Memory used: about **m**, not n.

# The m largest: a small example

m = 3, stream: **41 7 93 12 58 3 77**

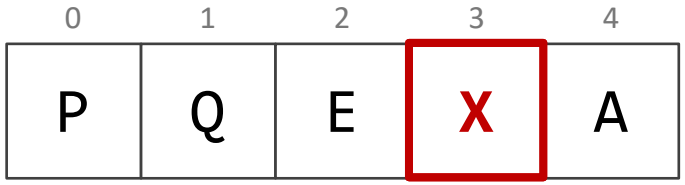
| read | delMin removes | PQ holds (smallest first) |
|---|---|---|
| 41 | | 41 |
| 7 | | 7 41 |
| 93 | | 7 41 93 |
| 12 | **7** | 12 41 93 |
| 58 | **12** | 41 58 93 |
| 3 | **3** | 41 58 93 |
| 77 | **41** | 58 77 93 |

At the end, the PQ holds **58, 77, 93**, the three largest.

It never held more than 4 numbers.

No matter how many numbers are in the stream, it will *still* never hold more than 4 numbers.

# Attempt 1: an unordered array

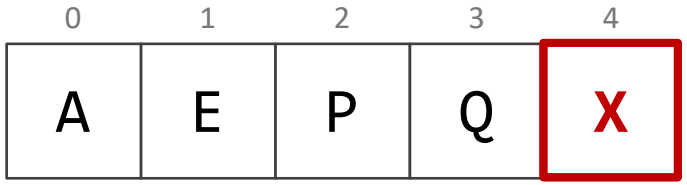



**insert:** put it at the end. **1 step.**

**delMax:** scan every item to find the largest (outlined), swap it to the end, and remove it. **n steps.**

| implementation | insert | delMax |
|---|---|---|
| unordered array | 1 | n |

# Attempt 2: an ordered array




**insert:** slide it left into place, like one pass of insertion sort. **Up to n steps.**

**delMax:** the largest is at the end (outlined). Take it. **1 step.**

| implementation | insert | delMax |
|---|---|---|
| unordered array | 1 | n |
| ordered array | n | 1 |

Linked lists give the **same** costs, sorted or not.

# In both cases, one operation is slow

| implementation | insert | delMax |
|---|---|---|
| unordered array | 1 | n |
| ordered array | n | 1 |
| **what we want** | **log n** | **log n** |

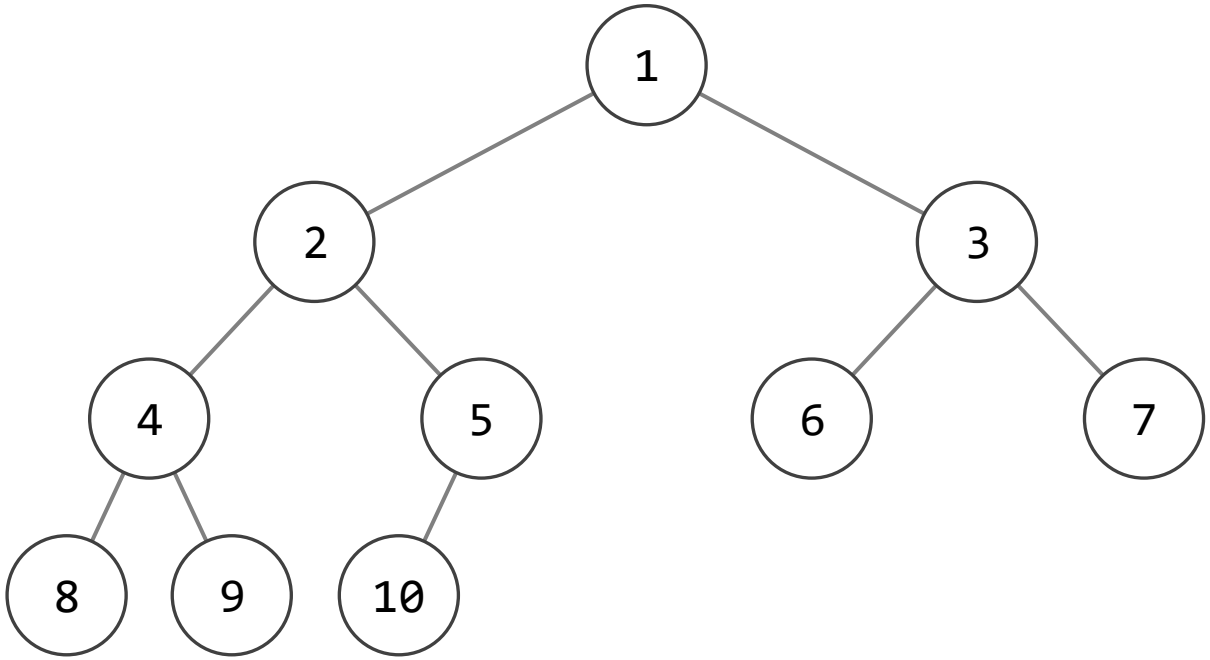
With either array, **one** operation is fast, but the **other** operation takes n steps.

For the m largest of n, that's about **n** × **m** steps in total for the slow operation.

Sorting all n first with quicksort is n log n, but needs **all n in memory**.

The **binary heap** does both operations in about **log n** steps, **without** using more than **m** memory.

# A complete binary tree



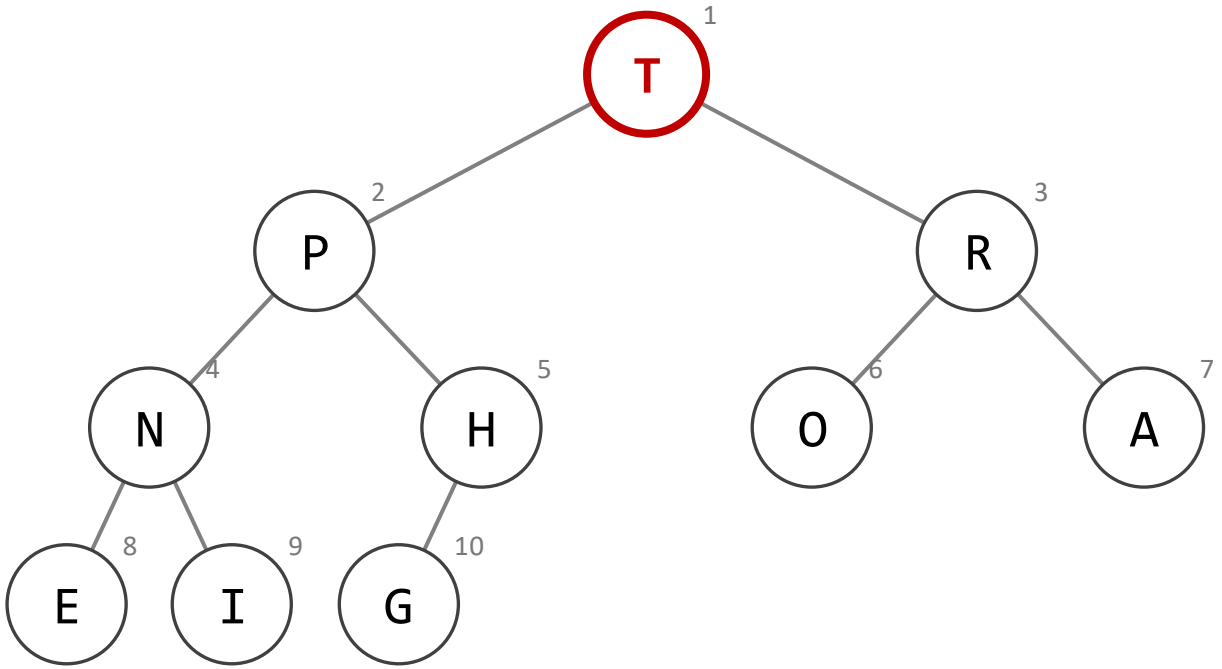

**Binary tree:** each node has at most two children.

**Complete:** every level is full, except maybe the bottom, which fills **from the left**.

Number the positions **level by level, left to right**, like reading a page.

| **n** | **height $\lfloor \lg n \rfloor$** |
|---|---:|
| 10 | 3 |
| 1,000 | 9 |
| 1,000,000 | 19 |

# Heap order




**Heap order:** every parent is ≥ each of its children.

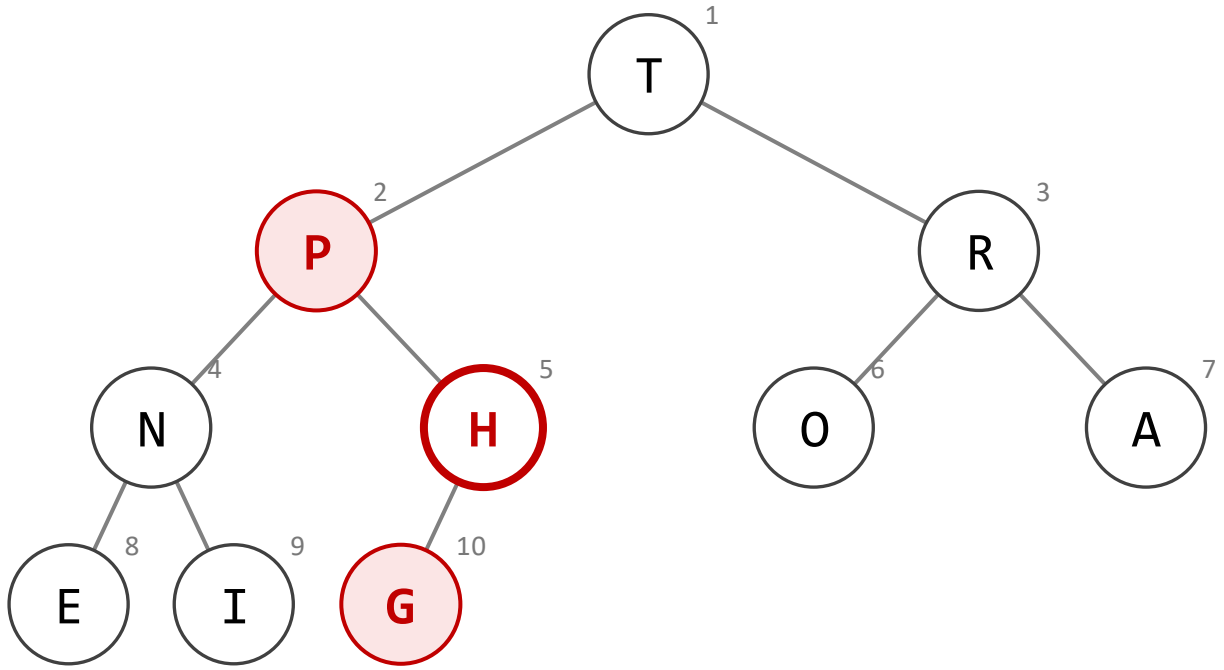
So the **largest** item is always at the **root** (outlined).

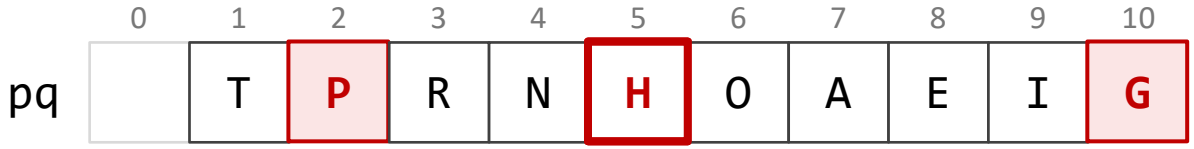
Heap order says nothing about left vs. right. A heap is **not sorted**.

This is the example heap from the book.

terminology: the **top** of a tree is the root
(yes, computer scientists draw trees upside-down)

# Storing the tree in an array




Item at position k goes in `pq[k]`. `pq[0]` is **unused**.

For the item at position **k**:

- parent: **k / 2** (integer division)
- children: **2k** and **2k + 1**

Example: H is at **5**. Parent: 5 / 2 = **2** (P). Children: **10** (G) and **11** (none yet).

This numbering dates to 1590: Eytzinger's family-tree tables put person k's parents at 2k and 2k + 1.

# less and exch, heap version

```
1  private boolean less(int i, int j) {
2      return pq[i].compareTo(pq[j]) < 0;
3  }
4
5  private void exch(int i, int j) {
6      Key t = pq[i];
7      pq[i] = pq[j];
8      pq[j] = t;
9  }
```

The same two helpers as in the sorting videos.

The difference: they take **positions** in `pq[]`, not items.

`less(k / 2, k)` asks: is my **parent** smaller than me?

# Swim: when a child is too large

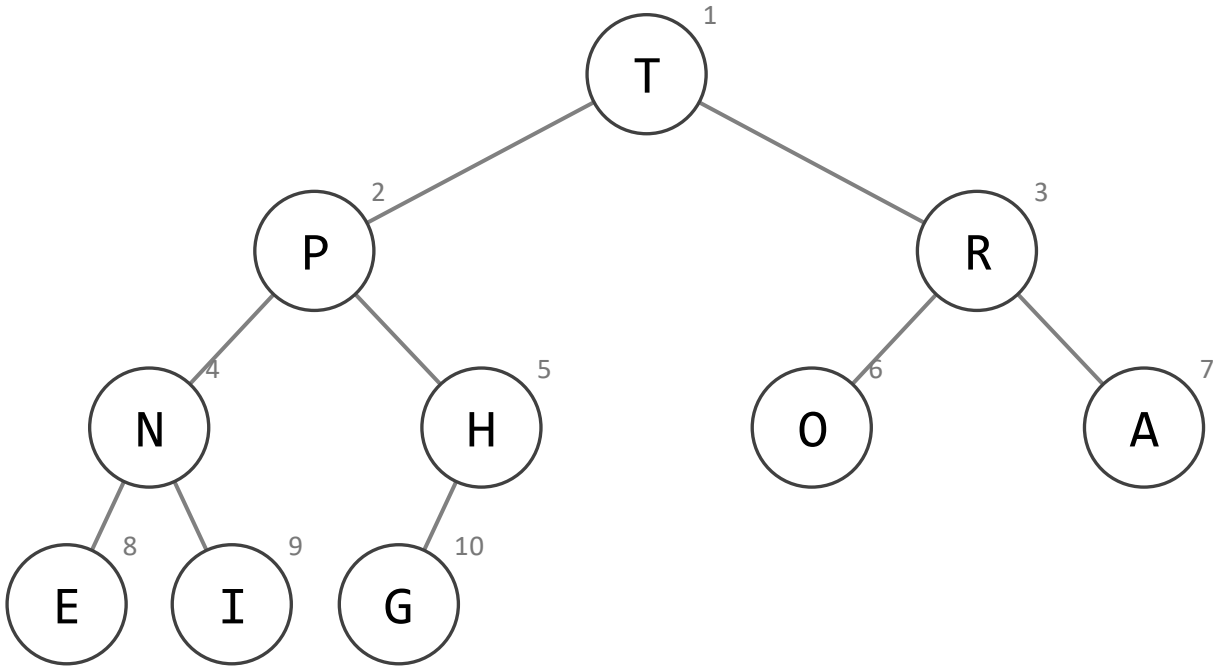
Suppose an item is **larger than its parent**. Heap order is broken, in one spot.

**Fix:** exchange it with its parent. Repeat until its parent is larger, or it reaches the root.

The item **swims** up the tree, one level per exchange.

**insert** uses swim: put the new item at the end, then swim it up.

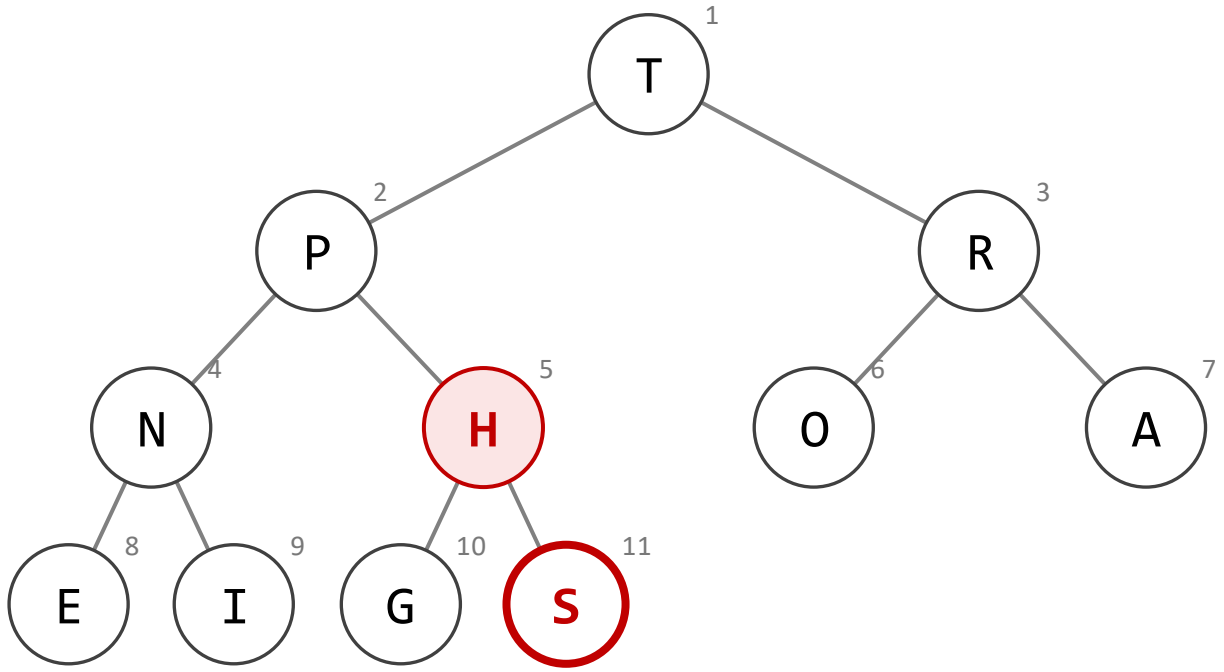
# insert(S)




| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| pq | | T | P | R | N | H | O | A | E | I | G |

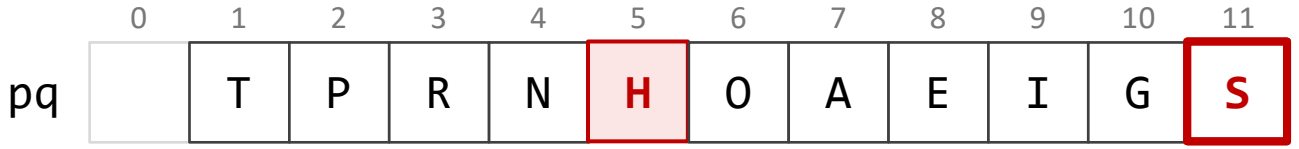
Our heap has **10 items**. Let's insert **S**.

# insert(S)




**pq[++n] = S:** S goes at the end, position 11. Now swim it up.

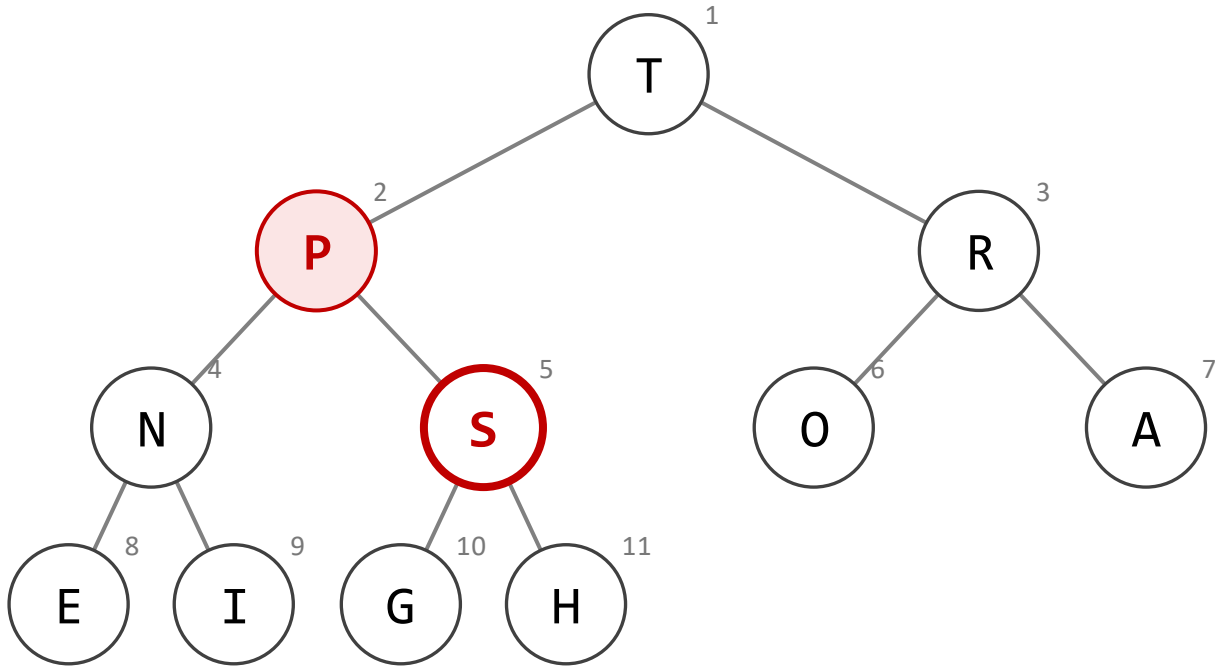
S is at position 11. Its parent is position 11 / 2 = 5, which holds H. Is H less than S? **Yes**: exchange them.



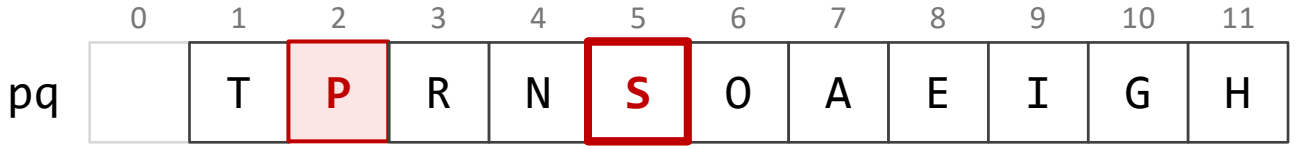

the item that's moving

its parent

# insert(S)



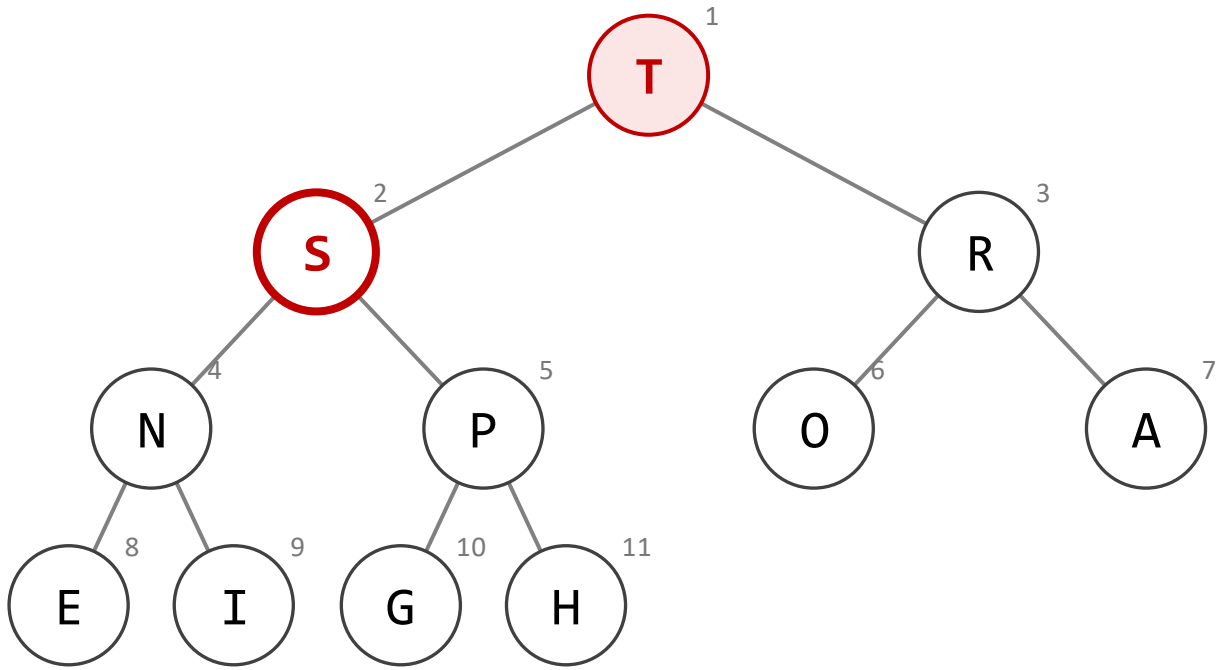

S is at position 5. Its parent is position 5 / 2 = 2, which holds P. Is P less than S?
**Yes**: exchange them.



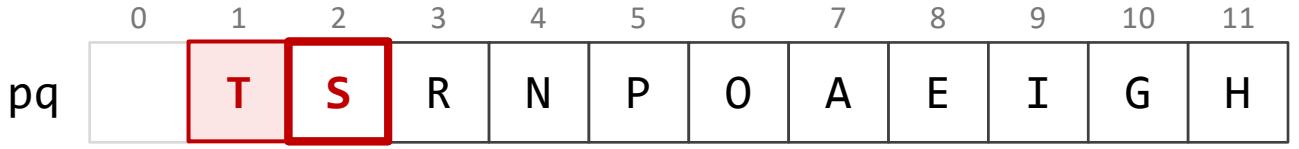

the item that's moving

its parent

# insert(S)



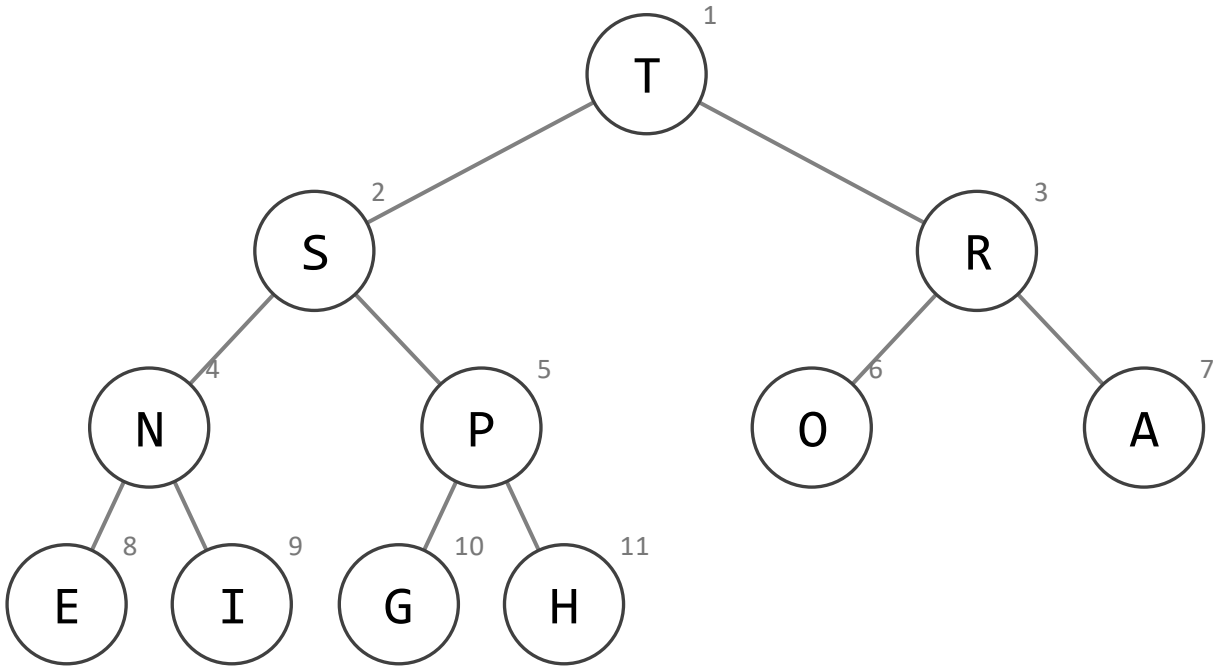

S is at position 2. Its parent is position 2 / 2 = 1, which holds T. Is T less than S? **No**, so S stops. Heap order is restored.




the item that's moving

its parent

# insert(S)




| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| pq | | T | S | R | N | P | O | A | E | I | G | H |

**Done.** S moved up **2 levels**, and heap order is restored.

Only the items on S's path to the root were touched.

# swim and insert: the code

```
1  private void swim(int k) {
2      while (k > 1 && less(k / 2, k)) {   // parent is smaller
3          exch(k / 2, k);
4          k = k / 2;
5      }
6  }
```

```
1  public void insert(Key v) {
2      pq[++n] = v;       // add v at the end
3      swim(n);           // and swim it up
4  }
```

`k > 1`: stop at the root.

`less(k / 2, k)`: is my **parent** smaller than me?

`pq[++n] = v` adds 1 to n **first**, then stores v there.

# Sink: when a parent is too small

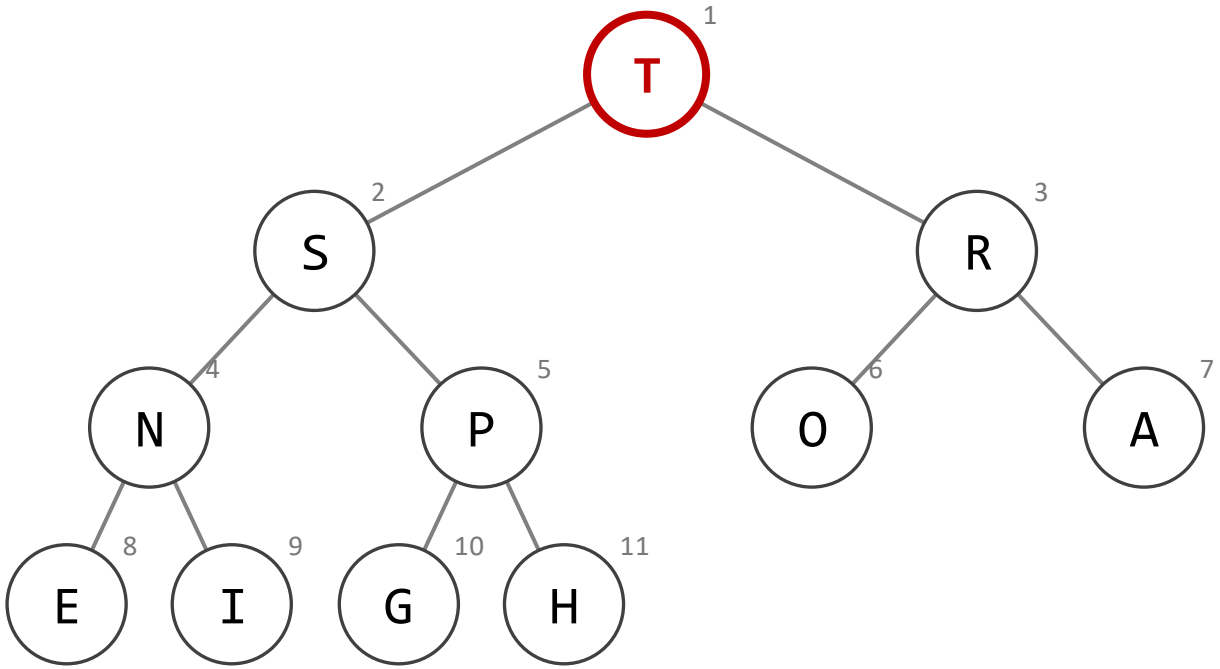
Suppose an item is **smaller than one of its children**.

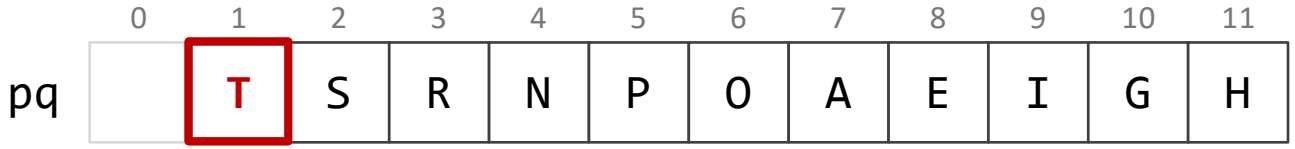
**Fix:** exchange it with its **larger** child. Repeat until it's at least as large as both children, or it has no children.

Why the **larger** child? It becomes the parent of the other one, so it has to be the bigger of the two.

The item **sinks** down the tree. **delMax** uses sink.

# delMax()

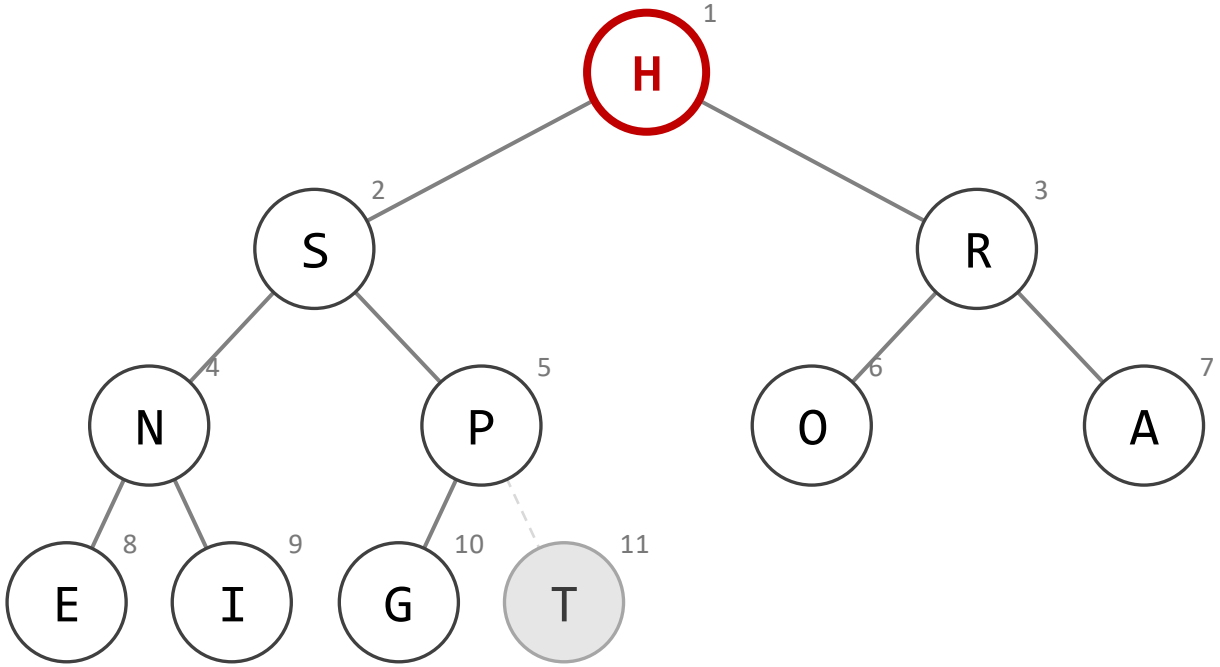


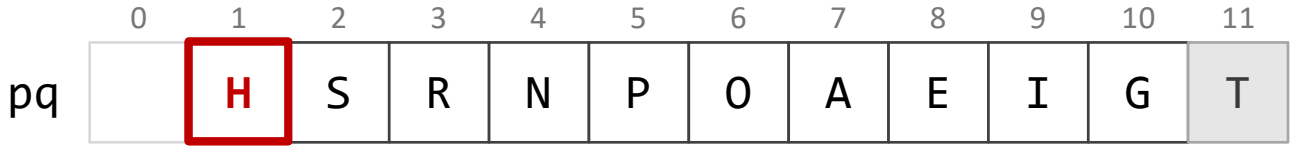




**delMax():** the largest item is always at the root: **T**.

We'll save it so we can return it at the end.

# delMax()



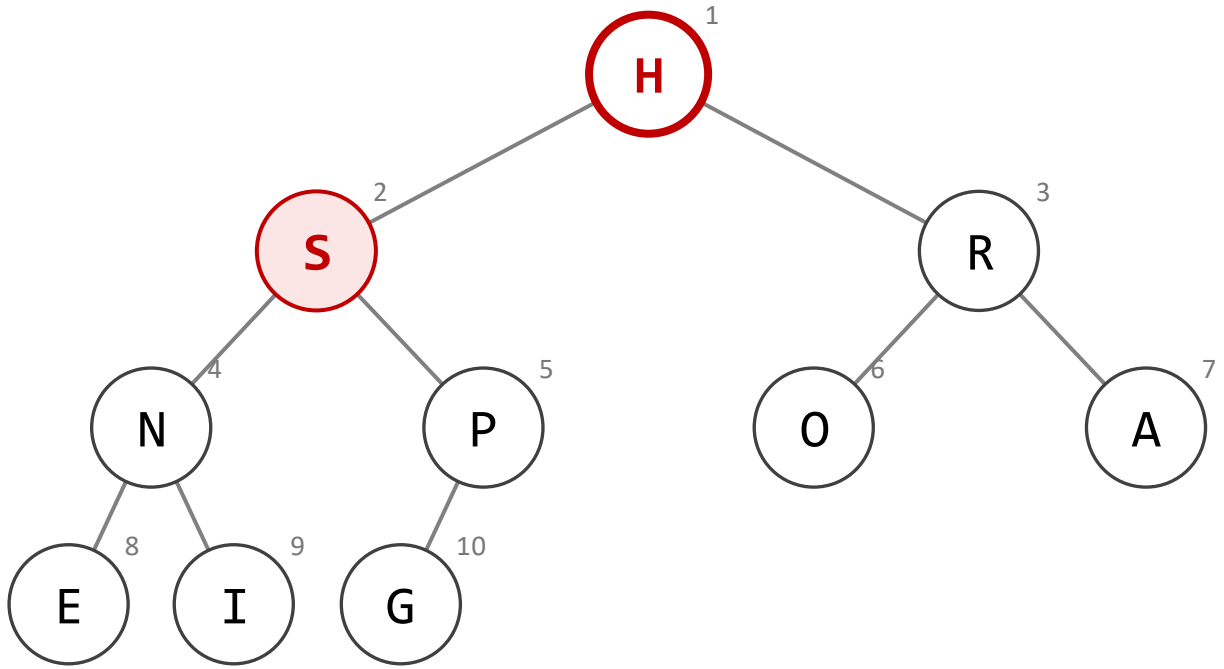




**exch(1, n--):** swap T with the last item, **H**. The heap now has **10** items.

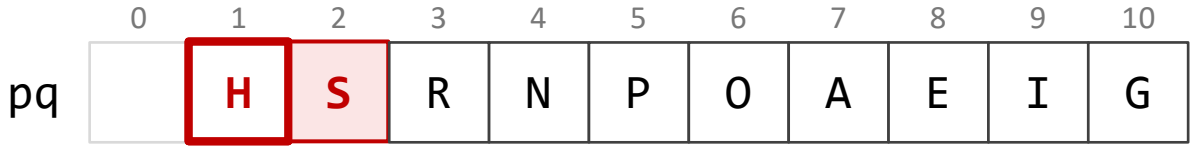
T (gray) is no longer in the heap. Next, `pq[11] = null` clears its slot.

H is at the root, and it's too small. Sink it.

# delMax(): sink(1)



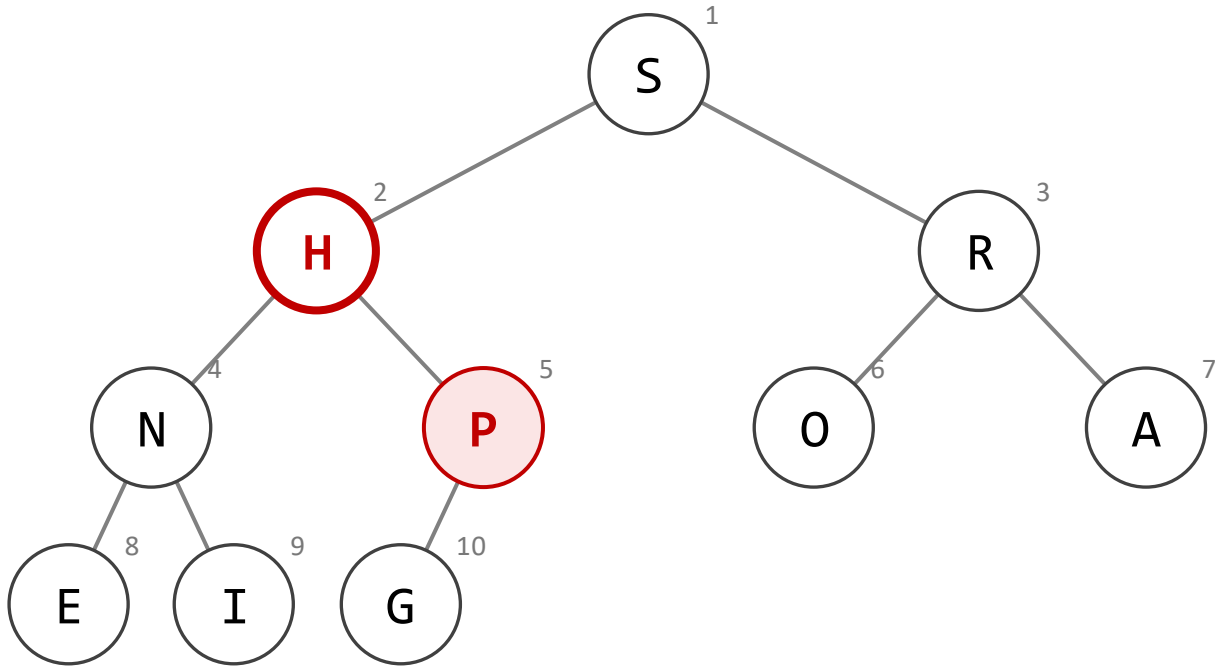

H is at position 1. Its children are positions 2 and 3: S and R. The larger is **S**. Is H less than S? **Yes**: exchange them.



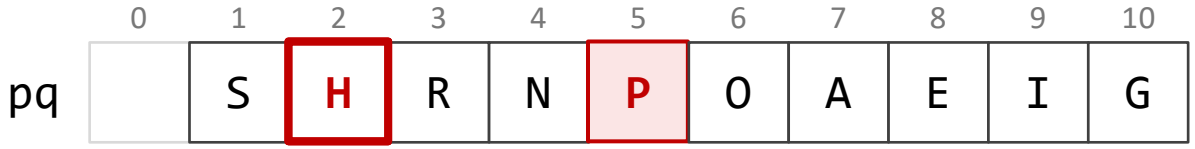

the item that's moving

its larger child

# delMax(): sink(1)



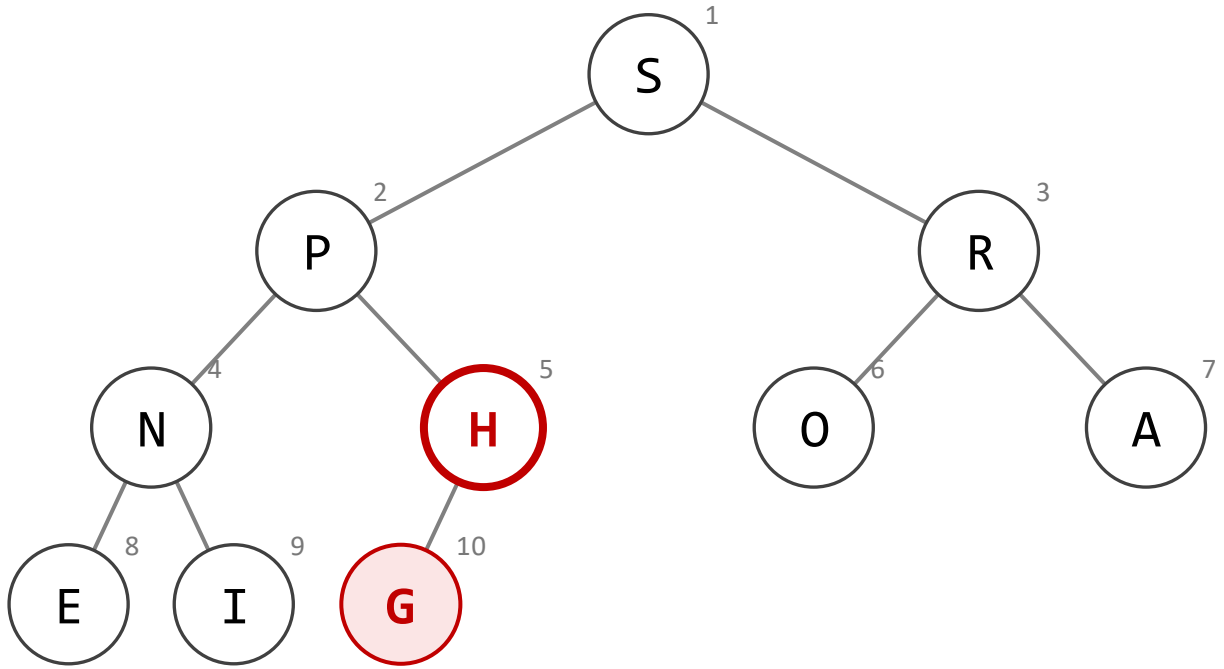

H is at position 2. Its children are positions 4 and 5: N and P. The larger is **P**. Is H less than P? **Yes**: exchange them.



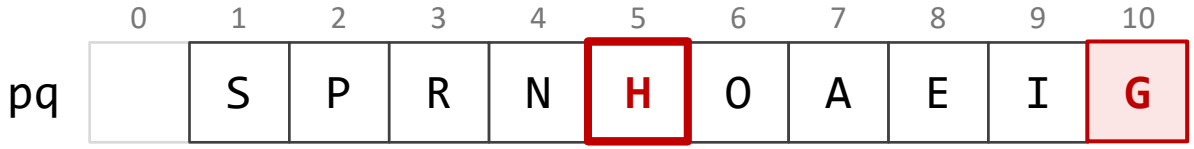

the item that's moving

its larger child

# delMax(): sink(1)



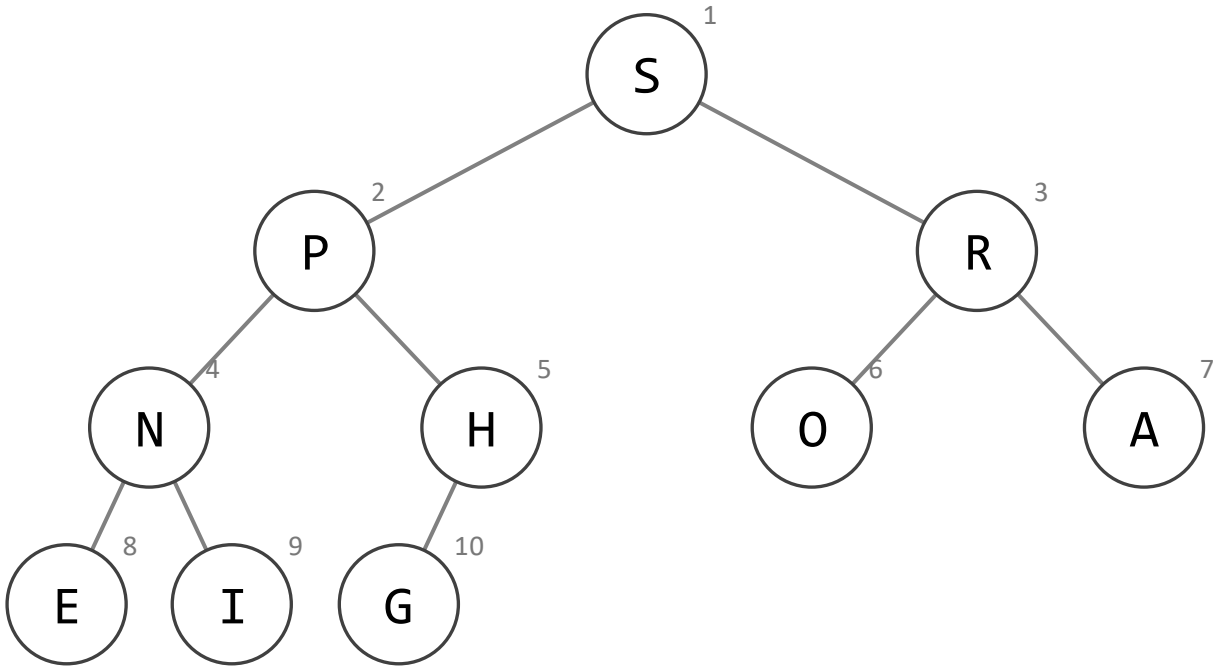

H is at position 5. Its only child is position 10: G. Is H less than G? **No**, so H stops. Heap order is restored.




the item that's moving

its larger child

# delMax()




| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| pq | | S | P | R | N | H | O | A | E | I | G |

**Done.** Return **T**.

The heap has 10 items, and the new largest, **S**, is at the root.

# sink and delMax: the code

```
1  private void sink(int k) {
2      while (2 * k <= n) {                // k has a child
3          int j = 2 * k;                  // left child
4          if (j < n && less(j, j + 1)) {
5              j++;                        // right child is larger
6          }
7          if (!less(k, j)) {
8              break;                      // heap order is OK
9          }
10         exch(k, j);
11         k = j;
12     }
13 }
```

```
1  public Key delMax() {
2      Key max = pq[1];       // the largest is at the root
3      exch(1, n--);          // swap it with the last item
4      pq[n + 1] = null;      // avoid loitering
5      sink(1);               // restore heap order
6      return max;
7  }
```

`j < n` checks that there **is** a right child before looking at it.

Lines 4–6 pick the **larger** child.

`exch(1, n--)` swaps with position n, **then** makes n one smaller.

`pq[n + 1] = null`: more on this in a few slides.

# What it costs

The tree has about **lg n** levels, and swim and sink move **one level** per exchange.

- **insert:** at most **1 + lg n** compares
- **delMax:** at most **2 lg n** compares (two per level: determine which child is larger, then compare with it)

| **implementation** | **insert** | **delMax** |
|---|---|---|
| unordered array | 1 | n |
| ordered array | n | 1 |
| **binary heap** | **log n** | **log n** |

| **m largest of n** | **time** | **memory** |
|---|---|---|
| sort everything | n log n | n |
| elementary PQ | n m | m |
| **binary heap** | **n log m** | **m** |

# The whole MaxPQ class

```
1  public class MaxPQ<Key extends Comparable<Key>> {
2      private Key[] pq;   // heap in pq[1..n]
3      private int n = 0;  // pq[0] is unused
4
5      public MaxPQ(int capacity) {
6          pq = (Key[]) new Comparable[capacity + 1];
7      }
8      public boolean isEmpty() { return n == 0; }
9      public int size() { return n; }
10     public Key max() { return pq[1]; }
11
12     public void insert(Key v) {
13         pq[++n] = v;
14         swim(n);
15     }
16     public Key delMax() {
17         Key max = pq[1];
18         exch(1, n--);
19         pq[n + 1] = null;
20         sink(1);
21         return max;
22     }
```

```
    private void swim(int k) {
        while (k > 1 && less(k / 2, k)) {
            exch(k / 2, k);
            k = k / 2;
        }
    }
    private void sink(int k) {
        while (2 * k <= n) {
            int j = 2 * k;
            if (j < n && less(j, j + 1)) j++;
            if (!less(k, j)) break;
            exch(k, j);
            k = j;
        }
    }
    private boolean less(int i, int j) {
        return pq[i].compareTo(pq[j]) < 0;
    }
    private void exch(int i, int j) {
        Key t = pq[i]; pq[i] = pq[j]; pq[j] = t;
    }
}
```

Line 6: Java won't allow new `Key[...]`, so we make a `Comparable` array and cast it. Eclipse warns "unchecked cast". That's expected.

# A few practical details

**Empty queue:** `delMax()` on an empty PQ should throw an exception (`NoSuchElementException`), not return garbage.

**Full array:** double it, like the resizing-array stack. Still about **log n** per operation, on average.

**Don't change an item while it's in the PQ.** The heap can't tell it changed, so heap order breaks with no error. Use immutable keys, like `String` and `Integer`.

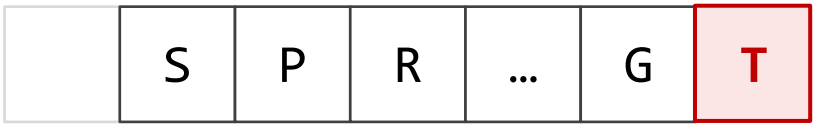
# Why pq[n + 1] = null? Loitering

Java frees an object's memory once **nothing in the program can reach it**.

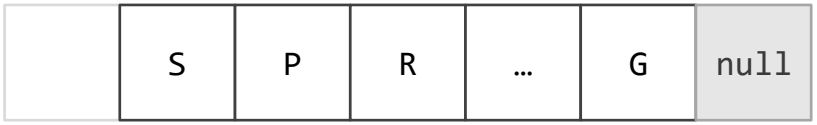
After `delMax()`, the heap is done with the item. But the old slot `pq[n + 1]` **still points to it**, so Java thinks it's still in use.

Its memory is never freed. That's called **loitering**.

**Fix:** set the slot to `null`. You only need this for array slots the data structure has **stopped using**.




before: the old slot still holds T




after `pq[n + 1] = null`

# Coming up: heapsort

Put n items into a heap, then call **delMax** n times: they come out **largest first**.

**Heapsort** does exactly that, inside the original array, with **no extra array**.

It reuses **sink**, almost line for line.